Numerical Analysis

Qingdao University of Technology

Team_7

October 2022

Question review

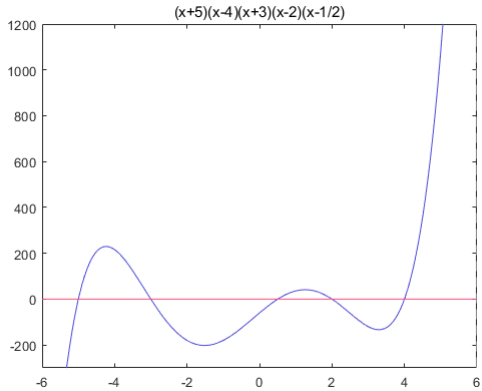
Design two nonlinear algebraic equations (degree at least 5 , with no less than 3 zeros) and find all their zeros (error less than 0.001).

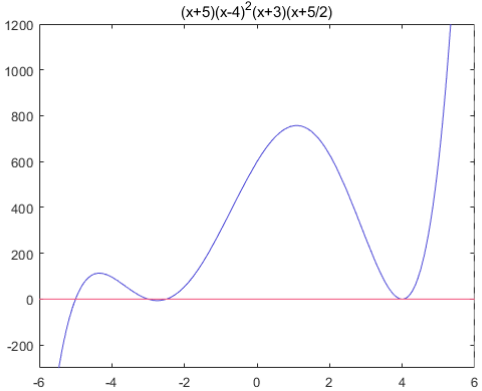
So,we firstly design the quintic algebric equations as follows

$$\begin{aligned} y_1 &= (x+5)(x-4)(x+3)(x-2)(x-\frac{1}{2}) \\ y_2 &= (x+5)(x-4)^2(x+3)(x+\frac{5}{2}) \end{aligned} \tag{1}$$

Figure

we use the Matlab to print the figure of the equation(1)





Detail_1

By analyzing the y_1 images, we can easily the minimum distance of zero is less than $\frac{1}{2}$, and the minimum distance of zero in y_2 is less than $\frac{1}{2}$ too.

Thus, We divide the whole interval$[-6, 6]$ into a number of cells with a length of $1/2$,and using the method of bisection solve for zero separately.

```
x0 = zeros(1,5);
k = 0;
L = 1/2;%分割长度
x0 = unique(x0);
 for i = -6:L:6
    if(y(i)*y(i+L)<=0)
        k = k + 1;
        x0(k) = f_Bis(y,i,i+L,0.001);%k 为第 k 的零点
    end
 end
fprintf("零点的横坐标为:")
```

```
%二分法具体函数
function z=f_Bis(f,a,b,accurate)
p=(a+b)/2;
n=0;
z=0;
```

```
while (abs(f(p)-0)>accurate)
    if(f(p)==0)%判断端点值是否为零点
        z=p;
        break;
    end
    if(f(a)==0)
```

```
        z=a;
        break;
    end
    if(f(b)==0)
        z=b;
        break;
    end
    if(f(a)*f(p)<0)
        b=p;
        p=(a+b)/2;
    else
        a=p;
        p=(a+b)/2;
    end
    z=p;
    n=n+1;
end
n;
end
```

Detail_2

Because of the existence of multiple zeros,Therefore, when solving the zeros, we should also consider the order of the zeros.

We take the derivative of the function, we get the first derivative of the function, and then we determine whether the zero at the first derivative is zero, and if it's zero, then it's the second zero, and so on.

zeros	-5.0000	-3.0000	-2.500	4.0000
order	1.0000	1.0000	1.0000	2.0000

```
%判断零点个数
function B = Judge(y,x0)
[C,R] = size(x0);
k = 1;
syms x
y1 = (x+5)*(x-4)*(x+3)*(x-2)*(x-1/2);
y=(x+5)*(x-4)^2*(x+3)*(x+5/2);%该函数只
    能键入，用参数输入时无法识别
f = y;
dy = diff(f)%求一阶导
ddy = diff(f,2);求二阶导
fprintf("函数的一阶导数值:")
dy_x = subs(dy,x,x0)
dy_x =double(dy_x);%强制转换
fprintf("函数的二阶导数值:")
ddy_x = subs(ddy,x,x0)
```

```
ddy_x = double(ddy_x);
for i=1:R
    if(dy_x(i) == 0)
        x0(2,k) = 2;
        if(ddy_x(i) == 0)
            x0(2,k) = 3;
        end
    else%此处只判断到三阶零点
    x0(2,k) = 1;
    end
    k = k + 1;
end
B = x0;
end
```

Question review

Find the numerical solution of the following nonlinear equations using iteration method and Newton method respectively and show your analysis of error by using norms of vector and matrix and demonstrate your computer code

$$\begin{cases} x_1^2 - 10x_1 + x_2^2 + 8 = 0 \\ x_1 x_2^2 + x_1 - 12x_2 + 7 = 0 \end{cases}$$

Fixed-point iteration method:

We have the following system of equations $\begin{cases} y_1 = f_1(x1, x2) \\ y_2 = f_2(x1, x2) \end{cases}(2)$,we set the initial point $P_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$,the iterative formula is

$$\begin{aligned} x_1^{(n+1)} &= f_1(x_1^{(n)}, x_2^{(n)}) \\ x_2^{(n+1)} &= f_2(x_1^{(n)}, x_2^{(n)}) \end{aligned} \tag{3}$$

In order to compare with the Newton-Raphson iteration method,The error is

$$\left\| \begin{bmatrix} x_1^{(n+1)} \\ x_2^{(n+1)} \end{bmatrix} - \begin{bmatrix} x_1^{(n)} \\ x_2^{(n)} \end{bmatrix} \right\| = \left\| \begin{bmatrix} f_1(x_1^n, x_2^n) - f_1(x_1^{(n-1)}, x_2^{(n-1)}) \\ f_2(x_1^n, x_2^n) - f_2(x_1^{(n-1)}, x_2^{(n-1)}) \end{bmatrix} \right\|$$

```
N = 100;%设置最大迭代次数
epsilon = 1e-9;%设置迭代的精度
%Fixed-point iteration
x(1,:) = [1;1];%迭代初始点
for i = 1:N
    x(i+1,1) = 1/10*(x(i,1)^2+x(i,2)^2+8);
    x(i+1,2) = 1/12*(x(i,1)+x(i,1)*x(i,2)^2+7);
    dx_Fp(i,:) = x(i+1,:) - x(i,:);
    if(sum(dx_Fp(i,:).^2)< epsilon)
        break;
    end
end
x_Fp = x
```

Newton-Raphson Iteration

1. Firstly,we set the initial point $x_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$,the $x^{(k+1)} - x^{(k)} = \frac{-F(x^{(k)})}{J(x^{(k)})}$.we set the $d^{(k)} = x^{(k+1)} - x^{(k)}$.
2. Secondly,The termination condition of the iteration is $\left\| d^{(k)} \right\| < \varepsilon$

```
%Newton-Raphson iteration
syms x1 x2;
f1(x1,x2) = x1^2 - 10*x1 + x2^2 + 8;
f2(x1,x2) = x1*x2^2 + x1 - 12*x2 + 7;

F(x1,x2) = [f1;f2];

x_solve(:,1) = [1;1];%迭代初始点

J(x1,x2) = jacobian(F,[x1 x2]);

dx_Nr =[0;0];          %X(n+1)-X(n) = d(X); 第一行表示第一次迭代 X(x1,x2) 值与前一值的插值
for i = 1:N                  % J*d(x) = -F(X) => d(x) = J  -F(x)
    dx_Nr(:,i) = J(x_solve(1,i),x_solve(2,i))\(-F(x_solve(1,i),x_solve(2,i)));
    if(sqrt(sum(dx_Nr(:,i).^2)) < epsilon) %
        x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
        break;
    else
        x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
```

```
    end
end
dx_Nr = dx_Nr';
x_Nr = x_solve'
```

由于 $A \setminus B$ 相当于 $A^{-1}B$

```
>> help \
 mldivide - 求解关于 x 的线性方程组 Ax = B
    此 MATLAB 函数 对线性方程组 A*x = B 求解。矩阵 A 和 B 必须具有相同的行数。如果 A
    未正确缩放或接近奇异值，MATLAB 将会显示警告信息，但还是会执行计算。

    x = A\B
    x = mldivide(A,B)
```

x_Fp = 10×2

1.0000	1.0000	%误差
1.0000	0.7500	0.0625
0.9563	0.7135	0.0032
0.9424	0.7036	2.9203e−04
0.9383	0.7007	2.4534e−05
0.9371	0.6999	2.0193e−06
0.9368	0.6997	1.6524e−07
0.9367	0.6996	1.3499e−08
0.9367	0.6996	1.1022e−09
0.9367	0.6996	8.9990e−11

%迭代 9 次

x_Nr = 6×2

1.0000	1.0000	
0.9211	0.6842	0.3255
0.9366	0.6995	0.0218
0.9367	0.6996	9.2006e−05
0.9367	0.6996	1.6056e−09
0.9367	0.6996	5.7481e−17

%迭代 5 次

% it is easy to find that the Newton method is better than fix-point method

Application promotion

We generalize the two-dimensional model to the three-dimensional model

$$\begin{cases} 5x_1 - x_2^2 + x_1x_3 = 2 \\ x_1x_3 + 4x_2 - x_3 = 0 \\ x_1^2 + x_2 - 3x_3 = 1 \end{cases}$$

,we only need to expand a variable， The changed details are as follows

```
%Fixed-point iteration
x(1,1) = 1;x(1,2) = 1;x(1,3) =1;%迭代初始点
for i = 1:N
    x(i+1,1) = 1/5*(x(i,2)^2-x(i,1)*x(i,3)+2);
    x(i+1,2) = 1/4*(x(i,3)-x(i,1)*x(i,3));
    x(i+1,3) = 1/3*(x(i,1)^2+x(i,2)-1);
    dx_Fp(i,:) = x(i+1,:) - x(i,:);
    if(sum(dx_Fp(i,:).^2)< epsilon)
        break;
    end
end
x_Fp = x
```

```
%Newton-Raphson iteration
syms x1 x2 x3;
f1(x1,x2,x3) = 5*x1-x2^2+x1*x3-2;
f2(x1,x2,x3) = x1*x3+4*x2-x3;
f3(x1,x2,x3) = x1^2+x2-3*x3-1;
```

```
F(x1,x2,x3) = [f1;f2;f3];

x_solve(:,1) = [0;0;0];%迭代初始点

J(x1,x2,x3) = jacobian(F,[x1 x2 x3]);

dx_Nr =[1;1;1];          %X(n+1)-X(n) = d(X); 第一行表示第一次迭代 X(x1,x2) 值与前一值的差值
for i = 1:N                  % J*d(x) = -F(X) => d(x) = J  -F(x)
    dx_Nr(:,i) = J(x_solve(1,i),x_solve(2,i),x_solve(3,i))\(-F(x_solve(1,i),
        x_solve(2,i),x_solve(3,i)));
    if(sqrt(sum(dx_Nr(:,i).^2)) < epsilon)
        x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
        break;
    else
        x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
    end
end
dx_Nr = dx_Nr';
x_Nr = x_solve'
```

Question Review

(1)Generate matrix $A(50 \times 50)$ and vector $b(50 \times 1)$ by using random number, and solve the linear system $AX = b$ by using Gaussian partial pivoting elimination algorithm.

Analyse

There are two important points:
1)Generate A nonsingular matrix to make it solvable;
2)Using sort algorithm to find the partial pivot.

```
n=50;
while(1)
A=round(rand(n)*99)+1;   %生成 n 阶方阵，元素取值范围为 1-100
%判断矩阵是否奇异
if  d=det(A)
     break
end
end
b=round(rand(n,1)*99)+1;   %生成 n*1 向量，元素取值范围为 1-100
X=[n,1];
T=[A,b];
Temp=[n+1,1];      %用作交换两行

for  i=1:n
     for  j=i+1:n   %选取绝对值最大的为列主元
          if(abs(T(j,i))>abs(T(i,i)))
               max=T(j,i);
               Temp=T(i,i);
               T(i,i)=T(j,i);
               T(j,i)=Temp;
          end
     end

     for  k=i+1:n   %计算得到上三角矩阵
     m_ik=T(k,i)/T(i,i);  %计算系数
     T(k,i+1:n+1)=T(k,i+1:n+1)-(T(i,i+1:n+1).*m_ik);
     T(k,i)=0;
     end
end

X(n,1)=T(n,n+1)/T(n,n);    %计算 x[n,1]
for  i=2:n   %计算 x[1:n-1,1]
     X(n-i+1,1)=(T(n-i+1,n+1)-T(n-i+2,n-i+2:n)*X(n-i+2:n,1))/T(i,i);
end
```

Question Reviwe

Randomly generate tridiagonal matrix (30 * 30), and use Jacobi iteration and Gauss Seidel iteration to solve them respectively. The error of the solution in terms of norm is required to be less than 1/1000.

Analyse

The most important premise to solve the problem is we can get a matrix which match the Jacobi iterative and Gauss-Seidel iterative conditions.

A strictly diagonally dominant is eligibility.

```
%创建 30 阶随机方阵，元素取值 1-100
A=round(rand(30)*99)+1;
D=zeros(30);
U=zeros(30);
L=zeros(30);
B=round(rand(30,1)*99)+1;

%将 D-U-L 变为严格对角占优矩阵，使得满足迭代条件
for j=1:29
    U(j,j+1)=-A(j,j+1);
    L(j+1,j)=-A(j+1,j);
end
D(1,1)=sum(abs(A(1,1:2)))+10*rand(1);
for i=2:29
    D(i,i)=sum(abs(A(i,i-1:i+1)))+10*
        rand(1);
end
D(30,30)=sum(abs(A(30,29:30)))+10*rand
    (1);

T=((D-L)^(-1))*U;
C=(D-L)^(-1);

e=max(abs(eig(T)));   %检查系数矩阵特征值

Xn=zeros(30,1);   %存储当前的迭代结果
Xn_1=ones(30,1);   %存储上一次的迭代结果
while sqrt(sum(abs(Xn-Xn_1).^2))>0.001
    Xn_1=Xn;
    for j=1:30
    Xn(j,1)=T(j,:)*Xn+C(j,:)*B;
    end
    fprintf('Xn = %f\n',Xn);
end
```

Question Reviwe

Use appropriate polynomial interpolation to approximate the following ellipse.
(1)$x^2 + y^2 = 5^2$;
(2)$\rho(\theta) = 2(1 - sin\theta)$(心形线).

Analyse

It's useful that using Spline Interpolation for the curve which is differentiable.But

(1)There are some point whose derivative are infinity for $f(x)$.We can divide the cuvre into several parts which includes one special point at least.And exchanging independent variable and dependent variable make some special points differentiable.

(2)Some points are nondifferentiable but continuous,thay can be the endpoints.

```
%对上下两段插值
x_i=-3:0.5:3;
y1=(25-x_i.^2).^(1/2);
y2=-(25-x_i.^2).^(1/2);
xx_i=-3:0.1:3;
yy1=spline(x_i,y1,xx_i);
yy2=spline(x_i,y2,xx_i);
%对左右两段插值
y_i=-4:0.5:4;
x1=(25-y_i.^2).^(1/2);
x2=-(25-y_i.^2).^(1/2);
yy_i=-4:0.1:4;
xx1=spline(y_i,x1,yy_i);
xx2=spline(y_i,x2,yy_i);
%画出图像
plot(xx_i,yy1,xx_i,yy2,xx1,yy_i,xx2,
```

```
yy_i);
```

```
    theta_1=0:pi/36:pi/2;
x=rho(theta_1).*cos(theta_1);
y=rho(theta_1).*sin(theta_1);
theta_11=0:pi/108:pi/2;
xx_1=rho(theta_11).*cos(theta_11);
yy_1=spline(x,y,xx_1);

theta_2=pi/2:pi/36:pi;
x=rho(theta_2).*cos(theta_2);
y=rho(theta_2).*sin(theta_2);
theta_22=pi/2:pi/108:pi;
xx_2=rho(theta_22).*cos(theta_22);
yy_2=spline(x,y,xx_2);

theta_3=pi:pi/36:(8*pi)/6;
x=rho(theta_3).*cos(theta_3);
y=rho(theta_3).*sin(theta_3);
theta_33=pi:pi/108:(8*pi)/6;
yy_3=rho(theta_33).*sin(theta_33);
xx_3=spline(y,x,yy_3);

theta_4=(8*pi)/6:pi/36:(10*pi)/6;
x=rho(theta_4).*cos(theta_4);
y=rho(theta_4).*sin(theta_4);
theta_44=(8*pi)/6:pi/108:(10*pi)/6;
xx_4=rho(theta_44).*cos(theta_44);
yy_4=spline(x,y,xx_4);

theta_5=(10*pi)/6:pi/36:2*pi;
x=rho(theta_5).*cos(theta_5);
y=rho(theta_5).*sin(theta_5);
theta_55=(10*pi)/6:pi/108:2*pi;
yy_5=rho(theta_55).*sin(theta_55);
xx_5=spline(y,x,yy_5);

plot(xx_1,yy_1,xx_2,yy_2,xx_3,yy_3,xx_4
    ,yy_4,xx_5,yy_5);
function rho=rho(theta)
```

```
rho=2.*(1-sin(theta));
end
```

We firstly assume the 上禾一秉 is x_1,the 中禾一秉 is x_2, the 下禾一秉 is x_3 and translate classical Chinese into mathematical equations,

$$\begin{cases} 3x_1 + 2x_2 + x_3 = 39 \\ 2x_1 + 3x_2 + x_3 = 34 \\ x_1 + 2x_2 + 3x_3 = 26 \end{cases} (4)$$

we use the Gauss-Seidel iteration to fix problems.the iteration formula is

$$X^{(k+1)} = T_G X^{(k)} + C_G$$

$T_G = (D - L)^{-1}U$ and $C_G = (D - L)^{-1}B$

```
    clc;
clear all;
close all;
%先假设上禾一乘为 x1，中禾一乘为 x2，下禾一乘为
    x3
%根据题意列出方程组
A = [3,2,1;2,3,1;1,2,3]
b = [39,34,26];
b = b'

fprintf("真实解为:")
if(A ~= 0 )
    X=(A\b)'
else%如果是奇异矩阵，有无穷解，则无法迭代
    fprintf("A is singular matrix,can't
        interation")
end

ab = [A,b];%a 为系数矩阵，b 为右端项
```

```
N = 100;%设置为迭代次数
Epsilon =1e-4;
[R,C] = size(A);
ab_D = A;
ab_L = A;
for i=1:R
    for j=1:R
        if(i~=j)
            ab_D(i,j) = 0;
        end
        if(i>j)
            ab_L(i,j) = -ab_L(i,j);
        else
            ab_L(i,j) = 0;
        end
    end
end

ab_U = ab_D - ab_L - A;
```

```
T_J = ab_D^-1*(ab_L + ab_U);
C_J = ab_D^-1*b;

T_G = (ab_D - ab_L)^-1 * ab_U;
C_G = (ab_D-ab_L)^-1*b;

X_G = [0;0;0];
X_J = [0;0;0];
error_G = 1;
error_J = 1;


fprintf("T_G:")
Split_G(A)
fprintf("T_J:")
Split_J(A)
```

```
%判断使用方法能否迭代
%利用 Jacobi 或者 Gauss 迭代法即可解出;
if(Split_G(A)<1)
    i = 1;
    while(error_G>=Epsilon)
        X_G(:,i+1) = T_G*X_G(:,i) + C_G
            ;
        error_G(i) = sqrt(sum((X_G(:,i
            +1)-X(1,:)').^2,1));
        i = i+1;
    end
    X_G = X_G';
    error_G
    X_G
else
    fprintf("Jacobi method can't
        convergent")
end
```

```
if(Split_J(A)<1)
    i = 1;
    while(error_J>=Epsilon)
        X_J(:,i+1) = T_J*X_J(:,i) + C_J
            ;
        error_J(i) = sqrt(sum((X_J(:,i
            +1)-X(1,:)').^2,1));
        i = i+1;
    end
    X_J = X_J';
    error_J
    X_J
else
    fprintf("Jacobi method  can't
        convergent")
end
```

```
function split = Split_J(A) %用来计算 |TJ|
[R,C] = size(A);
ab_D = A;
for i=1:R
    for j=1:R
        if(i~=j)
            ab_D(i,j) = 0;
        end
    end
end

split = abs(eig(ab_D^-1 * (ab_D-A)));
split = max(split);

end

function split = Split_G(A) %用来计算
    |TG|
[R,C] = size(A);
ab_D = A;
```

```
ab_L = A;
for i=1:R
    for j=1:R
        if(i~=j)
            ab_D(i,j) = 0;
        end
        if(i>j)
            ab_L(i,j) = -ab_L(i,j);
        else
            ab_L(i,j) = 0;
        end
    end
end

ab_U = ab_D - ab_L - A;

split = abs(eig((ab_D-ab_L)^-1 * ab_U))
    ;
split = max(split);

end
```

Result show

we use the Matlab to directly calculate the real solution,and use the iterative solution is compared with the real solution to get the error.

真实解为:

```
X = 1×3
    9.2500    4.2500    2.7500

T_G:
ans = 0.4730
T_J:
ans = 1.0000
error_G = 1×15
    4.0752    1.3142    0.5335    0.2393    0.1112    0.0523    0.0247    0.0117
              0.0055    0.0026    0.0012    0.0006    0.0003    0.0001    0.0001



X_G = 16×3
         0         0         0
   13.0000    2.6667    2.5556
   10.3704    3.5679    2.8313
    9.6776    3.9378    2.8156
```

```
    9.4363    4.1040    2.7853
    9.3356    4.1812    2.7673
    9.2901    4.2175    2.7583
    9.2689    4.2346    2.7539
    9.2589    4.2427    2.7519
    9.2542    4.2466    2.7509
    9.2520    4.2484    2.7504
    9.2509    4.2492    2.7502
    9.2504    4.2496    2.7501
    9.2502    4.2498    2.7500
    9.2501    4.2499    2.7500
    9.2500    4.2500    2.7500


Jacobi method  can't convergent
```

Conclusion

Thanks